BOSTON UNIVERSITY / CS 528 CLOUD COMPUTING

Homework 6

Database Normalization & Cloud-Based Predictive Modeling

Student: Zukhriddin Fakhriddinov Completed: October 31, 2026

Run ID: 20261031T162052Z GCP project: quick-formula-457319-q3

132,330	100.00%	33.59%
requests migrated	country test accuracy	income test accuracy

Executive summary

Homework 6 extends the HW5 PostgreSQL request-logging service with a losslessly migrated third-normal-form (3NF) schema and two predictive models trained on a short-lived Google Compute Engine VM. The first predicts country from client IP address; the second predicts income category from request and client attributes.

The country model exceeds the 99% requirement, with 100.00% accuracy on 26,420 held-out requests. The income model reaches 33.59% on 26,461 held-out requests, below the aspirational 35% target but well above the 12.60% majority-class baseline. This limitation is documented rather than hidden.

Submission evidence

Deliverable	Verified evidence
Database	132,330 request rows; 198 countries; 98,521 unique IPs; 1 preserved exception
Cloud prediction files	Two test prediction CSV files and metrics JSON were read back from GCS
Automation	VM created and deleted; Cloud SQL restarted and returned to STOPPED / NEVER
Source code	GitHub repository path listed on page 6; ensure hw6/ is pushed before submission

Data provenance: synthetic course-client HTTP request headers collected by HW5, not real geolocation or income measurements.

01 Database normalization

The original HW5 requests table stored country, client_ip, is_banned, age, gender, income and HTTP metadata together. The course specifies that a client IP maps to one country; the observed data also supports a consistent country-to-ban-status relationship. Hence the unnormalized design repeated values along these dependencies:

client_ip → country → is_banned

Because country and is_banned depend on a non-key attribute of an individual request, they are moved to dimension tables. A sparse exception table preserves the one legacy IP that occurred with two countries, so the original data can be reconstructed exactly without enforcing a false mapping.



Typical path: request -> IP -> country; exceptional original country stored sparsely.

New schema

Table	Keys / relevant attributes
hw6_countries	country_id PK; country UNIQUE; is_banned
hw6_ip_country	client_ip INET PK; country_id FK -> hw6_countries
hw6_requests	request_id PK; client_ip FK; gender, age, income, time_of_day, requested_file, status_code, received_at
hw6_request_country_exceptions	request_id PK/FK; original_country_id FK (sparse anomaly preservation)

The original failed_requests table is retained unchanged: it has a surrogate primary key and request-specific columns, with no evidenced non-key functional dependencies or reliable request foreign key in HW5.

Migration verification (after COMMIT)

132,330	0	0
source & migrated	field mismatches	country recon. errors

Additional checks: 198 distinct countries; 98,521 unique IPs; 6,145 legacy failure rows retained; one exception for 203.0.113.42/32 (observed as Iran and USA). Migration is transactional; original HW5 tables remain intact.

02 ML experiment design

Model 1 - IP address to country

Training reconstructs the source country by joining the normalized request, IP-country, country dimension and sparse exception tables. A numeric representation of the IPv4/IPv6 address is used as the sole predictor. The selected nearest_ip classifier finds the numerically closest observed address among the training IPs and returns its country label; no country-derived feature is used as an input.

The data is split by distinct IP address, so test IP addresses never appear in training. Three candidates (nearest IP, decision tree and ExtraTrees) are compared on a separate validation set. The model is fitted on the combined development split and evaluated only once on the held-out 20% group-based test set.

Model candidate	Validation accuracy	Validation macro-F1
Nearest IP (selected)	99.9858%	0.994841
Decision tree	99.9811%	0.994816
ExtraTrees	99.9858%	0.994863

Model 2 - request features to income

A CatBoost multiclass classifier predicts one of eight income categories. Its input attributes include country, age, gender, HTTP status, IP prefixes / bytes, requested file number, request timestamp components, and receipt timestamp components. Income itself is never included among the predictors.

The income experiment uses a stratified 64% training / 16% validation / 20% test row split with seed 5286. One bounded CatBoost configuration is evaluated: 500 iterations, depth 6, learning rate 0.075, L2 leaf regularization 5, early stopping after 65 non-improving rounds, and 2 threads. The best validation iteration was 474.

Split	Rows	Purpose
Train	84,672	Fit CatBoost
Validation	21,168	Early stopping and model check
Untouched test	26,461	Final reported income performance

Reproducibility: Python program hw6/step3_models.py runs on a newly created Debian 12 Compute Engine VM (e2-standard-2). Dependencies are listed in hw6/requirements_step3.txt. Results and prediction CSVs are written to GCS.

03 Held-out model results

IP -> Country **100.00%**

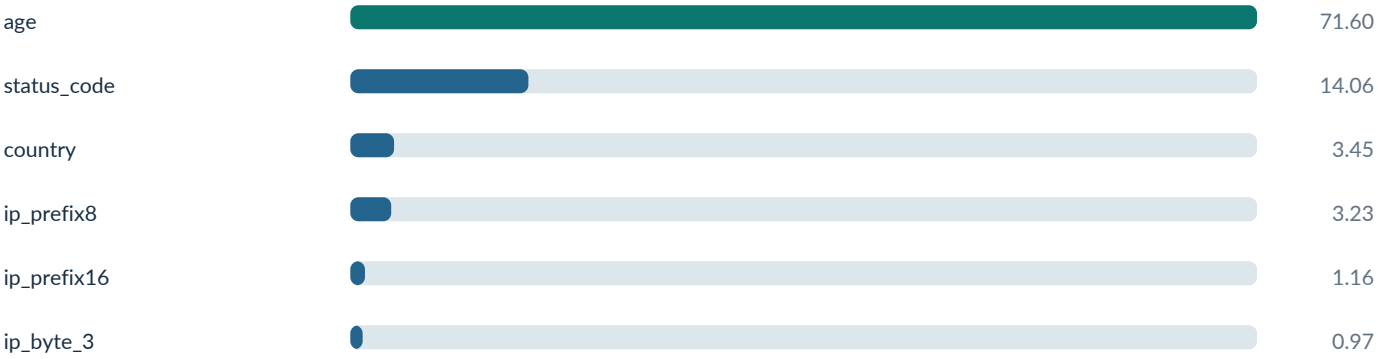
Target: 99% (vertical marker)

Features -> Income **33.59%**

Target: 35% (vertical marker)

Metric	Country predictor	Income predictor
Algorithm	Nearest numeric IP	CatBoost (depth 6)
Held-out test rows	26,420	26,461
Test accuracy	100.00%	33.59%
Balanced accuracy	100.00%	33.58%
Macro-F1	1.0000	0.3342
Assignment goal	At least 99%: achieved	Aim for 35%: below target

Income model - most influential features



Interpretation: age contributes the greatest importance (71.60), followed by HTTP status (14.06). Accuracy improves by about 21.0 percentage points over the 12.60% most-common-income baseline, but remains 1.41 percentage points below the instructor's 35% objective.

Limitations: the eight income bins are approximately evenly distributed, and the recorded features do not fully determine income. Results vary with the synthetic generator and random seed. We report the untouched test result (33.5891%), not the larger validation result (33.9616%), and do not tune against the test set.

04 Output artifacts and examples

The completed VM job uploaded both prediction CSVs and the machine-readable metrics, and the local orchestration script printed the metrics and first 12 rows of each CSV by reading them back from Cloud Storage. A success marker reported exit code 0.

`gs://cloud_hw2_bucket/hw6/runs/20261031T162052Z/`

Object	Content
country_test_predictions.csv	26,420 rows: request_id, client_ip, true_country, predicted_country, correct
income_test_predictions.csv	26,461 rows: request_id, true_income, predicted_income, correct
country_metrics.json	Country model evaluation/checkpoint
metrics.json	Combined model configurations, splits, accuracies, confusion summaries and feature importances
status.json	Success=true, exit_code=0; completed 2026-10-31 16:50:06 UTC
models.log / bootstrap.log / pip.log	Remote execution and dependency installation evidence

Sample country predictions (read back from the bucket)

Request ID	IP address	Actual country	Predicted country
7	241.9.152.72/32	Mexico	Mexico
23	3.53.149.136/32	Moldova	Moldova
24	134.22.152.113/32	Slovenia	Slovenia
29	180.130.205.207/32	Libya	Libya

Sample income predictions (read back from the bucket)

Request ID	Actual income	Predicted income	Correct
87514	10k-20k	10k-20k	True
40511	150k-250k	250k+	False
97462	100k-150k	60k-100k	False
18584	0-10k	0-10k	True

CSV entries here are illustrative rows from the terminal back-read; the complete per-request predictions are stored in the listed GCS objects.

05 Configuration, run and cleanup

Environment

Setting	Value
Project / zone	quick-formula-457319-q3 / us-central1-a
Database	Cloud SQL PostgreSQL 16: hw5-sql / hw5_requests
Machine	hw6-ml-1031162052, e2-standard-2, Debian 12, 30 GB balanced boot disk
VM service account	hw6-ml-sa@quick-formula-457319-q3.iam.gserviceaccount.com
Bucket	gs://cloud_hw2_bucket
Run ID	20261031T162052Z

How to reproduce from a terminal

Prerequisites: Google Cloud CLI authenticated to the project, Python 3 available locally, permissions to manage Compute Engine and Cloud SQL, and read/write access to the bucket. The scripts use the existing HW5 database and Secret Manager credential; no database password is committed to GitHub.

```
# Once: create/verify the normalized copy (preserves HW5 tables)
bash hw6/step2_migrate.sh

# Then: provision VM, train both models, print GCS outputs, clean up
bash hw6/step3_run_vm.sh

# Inspect saved predictions and final compute state
gcloud storage ls gs://cloud_hw2_bucket/hw6/runs/20261031T162052Z/
gcloud sql instances describe hw5-sql --project=quick-formula-457319-q3
```

Lifecycle and cleanup behavior

The Bash driver verifies that Cloud SQL starts in STOPPED / NEVER, creates or reuses VM permissions, uploads a code bundle to GCS, starts SQL safely, creates a short-lived VM, polls a GCS completion marker, and prints metrics and CSV preview rows from the bucket. Its EXIT trap deletes the temporary VM first and sets the SQL activation policy back to NEVER, even if training fails.

Recorded cleanup evidence	Final state
VM hw6-ml-1031162052	Deleted successfully
Cloud SQL stop operation 2e59bf27-...	DONE (2026-10-31 16:51:14 UTC)
Cloud SQL instance hw5-sql	STOPPED / NEVER

GitHub source link

https://github.com/zukhriddingit/cs528-hw4-vm-services/tree/main/hw6

The code folder must be pushed to the repository before the PDF is submitted; the GitHub link was not confirmed to contain hw6/ when this report was generated.

Appendix A Complete SQL schema and migration

The following are the complete executable SQL statements used by hw6/step2_migrate.py in its transactional migration: table creation, data transfer and indexes. Explanatory comments are omitted here for readability; the companion SQL file contains the fully annotated source.

```
CREATE TABLE IF NOT EXISTS hw6_countries (
    country_id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    country TEXT NOT NULL UNIQUE,
    is_banned BOOLEAN NOT NULL
);

CREATE TABLE IF NOT EXISTS hw6_ip_country (
    client_ip INET PRIMARY KEY,
    country_id INTEGER REFERENCES hw6_countries(country_id)
);

CREATE TABLE IF NOT EXISTS hw6_requests (
    request_id BIGINT PRIMARY KEY,
    client_ip INET NOT NULL REFERENCES hw6_ip_country(client_ip),
    gender VARCHAR(16),
    age VARCHAR(16),
    income VARCHAR(16),
    time_of_day TIMESTAMP WITHOUT TIME ZONE,
    requested_file TEXT NOT NULL,
    status_code SMALLINT NOT NULL,
    received_at TIMESTAMPTZ NOT NULL
);

CREATE TABLE IF NOT EXISTS hw6_request_country_exceptions (
    request_id BIGINT PRIMARY KEY REFERENCES hw6_requests(request_id),
    original_country_id INTEGER REFERENCES hw6_countries(country_id)
);

TRUNCATE TABLE
    hw6_request_country_exceptions,
    hw6_requests,
    hw6_ip_country,
    hw6_countries
RESTART IDENTITY;

INSERT INTO hw6_countries (country, is_banned)
SELECT country, BOOL_OR(is_banned)
FROM requests
WHERE country IS NOT NULL
GROUP BY country
ORDER BY country;

WITH ip_frequencies AS (
    SELECT client_ip, country, COUNT(*) AS n
    FROM requests
    GROUP BY client_ip, country
),

ranked AS (
    SELECT
        client_ip,
        country,
        ROW_NUMBER() OVER (
            PARTITION BY client_ip
            ORDER BY n DESC, country ASC NULLS LAST
```

```
        ) AS rn
    FROM ip_frequencies
)
INSERT INTO hw6_ip_country (client_ip, country_id)
SELECT ranked.client_ip, dim.country_id
FROM ranked
LEFT JOIN hw6_countries AS dim ON dim.country = ranked.country
WHERE ranked.rn = 1;

INSERT INTO hw6_requests (
    request_id, client_ip, gender, age, income,
    time_of_day, requested_file, status_code, received_at
)
SELECT
    request_id, client_ip, gender, age, income,
    time_of_day, requested_file, status_code, received_at
FROM requests;

INSERT INTO hw6_request_country_exceptions (request_id, original_country_id)
SELECT raw.request_id, original_dim.country_id
FROM requests AS raw
JOIN hw6_ip_country AS ip ON raw.client_ip = ip.client_ip
LEFT JOIN hw6_countries AS original_dim ON original_dim.country = raw.country
WHERE ip.country_id IS DISTINCT FROM original_dim.country_id;

CREATE INDEX IF NOT EXISTS hw6_requests_income_idx ON hw6_requests (income);

CREATE INDEX IF NOT EXISTS hw6_requests_received_at_idx ON hw6_requests (received_at);
```

Supporting artifacts: hw6/step2_3nf.sql, hw6/step2_migrate.py, hw6/step3_models.py, hw6/step3_vm_startup.sh, hw6/step3_run_vm.sh, and hw6/requirements_step3.txt. All reported results are from the provided October 31, 2026 terminal transcript, not synthetic demonstration outputs.